ISYE 6740, Summer 2025, Homework 3

100 points + 10 bonus points

Prof. Yao Xie

There are no gradescope requirements for this assigment. Submit all your code to canvas as usual, But feel free to code in any way you prefer.
Provided Data:

- Q2: Implementing EM for MNIST Dataset [data.dat, data.mat, label.mat, label.dat]
- Q4: Comparing classifiers: Divorce classification/prediction [marriage.csv]

All results that are present only in code/notebooks, but not in your PDF report will not be accepted for points.

1. Conceptual questions. [15 points]

1. (5 points) Please compare the pros and cons of KDE over histogram, and give at least one advantage and disadvantage to each.

   Solution:

   Both Kernel Density Estimation (KDE) and histograms are non-parametric methods used for estimating the probability density function (PDF) of a random variable. They provide visual representations of the distribution of a dataset. However, they differ in their approach and characteristics.

   Histograms: A histogram is a graphical representation of the distribution of numerical data. It partitions the data into a set of bins, and the height of each bar represents the number of data points falling into that bin (or the proportion of data points).

   - **Advantages of Histograms:**
     - **Simplicity and Interpretability:** Histograms are very easy to understand and interpret. The concept of counting observations within bins is intuitive, making them accessible to a wide audience.
     - **Computational Efficiency:** They are computationally less intensive to construct, especially for large datasets, as they primarily involve binning and counting.

- **Disadvantages of Histograms:**
  - **Binning Dependence:** The appearance of a histogram is highly dependent on the choice of bin width and bin origin. Different choices can lead to very different interpretations of the underlying distribution, potentially obscuring features or creating artificial ones.
  - **Lack of Smoothness:** Histograms produce a discontinuous, step-like representation of the density. This can be problematic when the true underlying distribution is smooth, as the histogram fails to capture this smoothness, especially with sparse data.
  - **Information Loss:** By grouping data into bins, histograms lose the individual data point information within each bin.

Kernel Density Estimation (KDE): KDE is a non-parametric method for estimating the PDF of a random variable. It works by placing a kernel function (typically a Gaussian) over each data point and then summing these kernel functions to obtain a smooth estimate of the density.

- **Advantages of KDE (over Histograms):**
  - **Smoothness:** KDE produces a smooth and continuous estimate of the density function, which is generally a more accurate representation of the underlying continuous distribution compared to the discrete bins of a histogram. This makes it better for visualizing the shape of the distribution without the blockiness of histograms.
  - **Less Dependent on Origin/Binning:** KDE is not subject to the artificial discontinuities and choices of bin edges that plague histograms. While it still depends on a bandwidth parameter, its effect on the shape of the estimate is often more intuitive and less prone to misleading interpretations than bin choices in histograms.
  - **Better for Small Sample Sizes:** For small datasets, KDE can provide a more informative and less jagged density estimate than a histogram.
- **Disadvantages of KDE (over Histograms):**
  - **Computational Complexity:** KDE can be more computationally intensive, especially for large datasets, as it involves calculating the sum of kernel functions for each evaluation point. This can be a significant drawback for real-time applications or very large datasets.
  - **Bandwidth Selection:** KDE requires the selection of a bandwidth parameter. The choice of bandwidth significantly affects the smoothness and accuracy of the density estimate. An overly small bandwidth can lead to an estimate that is too noisy (under-smoothing), while an overly large bandwidth can over-smooth the data, obscuring important features. Choosing an optimal bandwidth can be challenging.

– **Boundary Effects:** KDE can exhibit boundary effects, where the density estimate at the edges of the data range might be inaccurate, especially if not handled properly (e.g., using reflective or mirrored kernels).

In summary, KDE offers a smoother and potentially more accurate representation of the underlying density, especially for continuous data, but at the cost of higher computational complexity and the need for careful bandwidth selection. Histograms are simpler and more computationally efficient but are highly sensitive to binning choices and produce a less smooth representation.

2. (5 points) Explain why the maximum likelihood estimation (MLE) cannot be applied directly to estimate the parameters of a GMM. Additionally, what is the standard approach used to fit a GMM effectively?

Solution:

Why MLE cannot be applied directly to estimate GMM parameters:

A Gaussian Mixture Model (GMM) assumes that the data points are generated from a mixture of K Gaussian distributions. The parameters of a GMM include:

- The mixing coefficients (or prior probabilities) for each component, π_k for $k = 1, \ldots, K$, such that $\sum_{k=1}^{K} \pi_k = 1$ and $\pi_k \geq 0$.
- The mean vector for each component, μ_k for $k = 1, \ldots, K$.
- The covariance matrix for each component, Σ_k for $k = 1, \ldots, K$.

Given a dataset $X = \{x_1, x_2, \ldots, x_N\}$ of N independent and identically distributed (i.i.d.) observations, the likelihood function for a GMM is given by:

$$L(\Theta|X) = \prod_{i=1}^{N} p(x_i|\Theta) = \prod_{i=1}^{N} \left(\sum_{k=1}^{K} \pi_k \mathcal{N}(x_i|\mu_k, \Sigma_k) \right)$$

where $\Theta = \{\pi_k, \mu_k, \Sigma_k\}_{k=1}^{K}$ represents the set of all parameters, and $\mathcal{N}(x_i|\mu_k, \Sigma_k)$ is the probability density function of a multivariate Gaussian distribution.

The logarithm of the likelihood function (log-likelihood) is:

$$\log L(\Theta|X) = \sum_{i=1}^{N} \log \left(\sum_{k=1}^{K} \pi_k \mathcal{N}(x_i|\mu_k, \Sigma_k) \right)$$

The challenge in directly applying Maximum Likelihood Estimation (MLE) to this log-likelihood function arises from the presence of the sum inside the logarithm. This sum couples the parameters of different mixture components, making it impossible to analytically derive closed-form solutions for the parameters by setting the partial derivatives with respect to each parameter to zero.

Specifically, if we try to take the derivative with respect to, say, μ_k, we get terms involving $\frac{1}{\sum_j \pi_j \mathcal{N}(x_i|\mu_j,\Sigma_j)}$ which makes the equations highly non-linear and coupled. This leads to a situation where there is no straightforward analytical solution to find the parameters that maximize the likelihood. The direct maximization of this non-convex likelihood function is computationally intractable using standard optimization techniques, as it would involve solving a complex system of non-linear equations.

Furthermore, the likelihood function for GMMs can have multiple local maxima, and directly applying gradient-based optimization methods without a good initialization can lead to convergence to sub-optimal solutions. There's also the issue of singularities: if one of the covariance matrices Σ_k collapses to zero (e.g., if one component perfectly fits a single data point), the likelihood can tend to infinity, making the maximum likelihood estimate ill-defined in certain scenarios.

Standard approach used to fit a GMM effectively:

The standard and most effective approach used to fit a GMM is the **Expectation-Maximization (EM) algorithm**.

The EM algorithm is an iterative optimization algorithm for finding maximum likelihood or maximum a posteriori (MAP) estimates of parameters in statistical models, where the model depends on unobserved latent variables. In the context of GMMs, the unobserved latent variable for each data point x_i is the indicator variable z_i, which specifies which of the K mixture components generated x_i. That is, $z_i \in \{1, \ldots, K\}$, and $z_{ik} = 1$ if x_i was generated by component k, and 0 otherwise.

The EM algorithm for GMMs proceeds in two main steps, which are iteratively repeated until convergence:

(a) **E-step (Expectation Step):** In this step, given the current estimates of the model parameters $\Theta^{(t)} = \{\pi_k^{(t)}, \mu_k^{(t)}, \Sigma_k^{(t)}\}_{k=1}^K$, we compute the posterior probability (or responsibility) that each data point x_i belongs to each component k. This is the expectation of the latent variable z_{ik} given the observed data and current parameters. The responsibility τ_k^i (also denoted as $\gamma(z_{ik})$) for data point x_i belonging to component k is calculated using Bayes' rule:

$$\tau_k^i = P(z_{ik} = 1|x_i, \Theta^{(t)}) = \frac{\pi_k^{(t)} \mathcal{N}(x_i|\mu_k^{(t)}, \Sigma_k^{(t)})}{\sum_{j=1}^K \pi_j^{(t)} \mathcal{N}(x_i|\mu_j^{(t)}, \Sigma_j^{(t)})}$$

This step essentially assigns each data point a "soft" membership to each of the mixture components.

(b) **M-step (Maximization Step):** In this step, given the responsibilities calculated in the E-step, we update the model parameters to maximize the expected log-likelihood (which is the complete-data log-likelihood averaged over the posterior distribution of the latent variables). This step treats the responsibilities τ_k^i as if they were fixed "weights" for each data point in each component. The updated parameters $\Theta^{(t+1)}$ are calculated as follows:

- **Mixing Coefficients:**

$$\pi_k^{(t+1)} = \frac{N_k}{N} = \frac{\sum_{i=1}^{N} \tau_k^i}{N}$$

where $N_k = \sum_{i=1}^{N} \tau_k^i$ is the effective number of data points assigned to component k.

- **Means:**

$$\mu_k^{(t+1)} = \frac{\sum_{i=1}^{N} \tau_k^i x_i}{\sum_{i=1}^{N} \tau_k^i} = \frac{\sum_{i=1}^{N} \tau_k^i x_i}{N_k}$$

- **Covariance Matrices:**

$$\Sigma_k^{(t+1)} = \frac{\sum_{i=1}^{N} \tau_k^i (x_i - \mu_k^{(t+1)})(x_i - \mu_k^{(t+1)})^T}{\sum_{i=1}^{N} \tau_k^i} = \frac{\sum_{i=1}^{N} \tau_k^i (x_i - \mu_k^{(t+1)})(x_i - \mu_k^{(t+1)})^T}{N_k}$$

The E-step and M-step are repeated until the change in the log-likelihood or the parameters falls below a certain threshold, indicating convergence. The EM algorithm is guaranteed to converge to a local maximum of the likelihood function. Due to the possibility of local optima, it's common practice to run the EM algorithm multiple times with different random initializations of the parameters.

3. (5 points) For the EM algorithm for GMM, please show how to use the Bayes rule to derive τ_k^i in a closed-form expression.

Solution:

In the Expectation-Maximization (EM) algorithm for Gaussian Mixture Models (GMMs), the E-step involves calculating the posterior probability that a data point x_i belongs to a specific component k, given the current model parameters. This posterior probability is often referred to as the responsibility, denoted by τ_k^i (or sometimes $\gamma(z_{ik})$).

Let's define the terms:

- $P(x_i|\text{model parameters})$: The probability density of observing data point x_i given the current GMM parameters.
- $P(z_{ik} = 1)$: The prior probability that a data point comes from component k. In GMM, this is simply the mixing coefficient π_k.
- $P(x_i|z_{ik} = 1, \text{model parameters})$: The probability density of observing x_i given that it came from component k. This is given by the Gaussian probability density function $\mathcal{N}(x_i|\mu_k, \Sigma_k)$.

We want to find $\tau_k^i = P(z_{ik} = 1|x_i, \text{model parameters})$. Using Bayes' Rule, which states $P(A|B) = \frac{P(B|A)P(A)}{P(B)}$, we can write:

$$P(z_{ik} = 1|x_i, \text{model parameters}) = \frac{P(x_i|z_{ik} = 1, \text{model parameters})P(z_{ik} = 1)}{P(x_i|\text{model parameters})}$$

Let's break down each term:

1. **Prior probability of component k:**

$$P(z_{ik} = 1) = \pi_k$$

This is the mixing coefficient for the k-th component, representing the proportion of data points expected to be generated by that component.

2. **Likelihood of x_i given it came from component k:**

$$P(x_i|z_{ik} = 1, \text{model parameters}) = \mathcal{N}(x_i|\mu_k, \Sigma_k)$$

This is the probability density of x_i under the Gaussian distribution of component k, with mean μ_k and covariance Σ_k. The formula for a multivariate Gaussian PDF is:

$$\mathcal{N}(x|\mu, \Sigma) = \frac{1}{\sqrt{(2\pi)^D|\Sigma|}} \exp\left(-\frac{1}{2}(x-\mu)^T\Sigma^{-1}(x-\mu)\right)$$

where D is the dimensionality of the data.

3. **Marginal probability of x_i:** The denominator $P(x_i|\text{model parameters})$ represents the total probability density of observing x_i, considering all possible components. Since x_i must have come from one of the K components, we can sum the probabilities of x_i being generated by each component, weighted by their prior probabilities:

$$P(x_i|\text{model parameters}) = \sum_{j=1}^{K} P(x_i|z_{ij} = 1, \text{model parameters})P(z_{ij} = 1)$$

$$P(x_i|\text{model parameters}) = \sum_{j=1}^{K} \pi_j \mathcal{N}(x_i|\mu_j, \Sigma_j)$$

This sum represents the probability density of x_i under the entire mixture model.

Now, substituting these terms back into Bayes' rule, we get the closed-form expression for τ_k^i:

$$\tau_k^i = \frac{\pi_k \mathcal{N}(x_i|\mu_k, \Sigma_k)}{\sum_{j=1}^{K} \pi_j \mathcal{N}(x_i|\mu_j, \Sigma_j)}$$

This equation provides the responsibility τ_k^i, which is the posterior probability that data point x_i was generated by component k. This value is crucial in the E-step of the EM algorithm, as it quantifies how much each data point contributes to the estimation of the parameters of each Gaussian component in the subsequent M-step.

2. Implementing EM for MNIST dataset. [30 points]

Implement the EM algorithm for fitting a Gaussian mixture model for the MNIST handwritten digits dataset. For this question, we reduce the dataset to be only two cases, of digits "2" and "6" only. Thus, you will fit GMM with $C = 2$. Use the data file data.mat or data.dat. True label of the data are also provided in label.mat and label.dat.

The matrix images is of size 784-by-1990, i.e., there are 1990 images in total, and each column of the matrix corresponds to one image of size 28-by-28 pixels (the image is vectorized; the original image can be recovered by mapping the vector into a matrix).

First use PCA to reduce the dimensionality of the data before applying to EM. We will put all "6" and "2" digits together, to project the original data into 4-dimensional vectors.

Now implement EM algorithm for the projected data (with 4-dimensions).
(In this question, we use the same set of data from the provided data files for training and testing)

1. (10 points) Implement EM algorithm yourself. Use the following initialization

   The EM algorithm for GMM is an iterative optimization method.

   - **Initialization for mean:** For each of the $C = 2$ components, the mean vector $\boldsymbol{\mu}_k \in \mathbb{R}^4$ was initialized as a random Gaussian vector with zero mean. Specifically, each element was drawn from a standard normal distribution and scaled by a small factor (e.g., 0.1) to ensure initial means are close to the origin in the PCA space.
   - **Initialization for covariance:** For each component k, two Gaussian random matrices $S_k \in \mathbb{R}^{4\times 4}$ were generated, where each element of S_k was drawn from a standard normal distribution. The covariance matrix for component k, $\boldsymbol{\Sigma}_k$, was then initialized as $\boldsymbol{\Sigma}_k = S_k S_k^T + I_4$, where I_4 is the 4-by-4 identity matrix. This ensures that the initial covariance matrices are symmetric and positive definite.

   EM Algorithm Steps:

   E-Step (Expectation): Compute the responsibility τ_{ik} that component k takes for data point $\mathbf{x}_i$:

   $$\tau_{ik} = \frac{\pi_k \mathcal{N}(\mathbf{x}_i|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_{j=1}^{C} \pi_j \mathcal{N}(\mathbf{x}_i|\boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}$$

   where π_k are the current mixing coefficients, and $\mathcal{N}(\cdot)$ is the multivariate Gaussian PDF. A small regularization term ($10^{-6}I$) was added to the diagonal of $\boldsymbol{\Sigma}_k$ during PDF calculation for numerical stability.

   M-Step (Maximization): Update the GMM parameters based on the computed responsibilities:

- **Update Weights:**

$$\pi_k^{new} = \frac{\sum_{i=1}^{N} \tau_{ik}}{N}$$

- **Update Means:**

$$\boldsymbol{\mu}_k^{new} = \frac{\sum_{i=1}^{N} \tau_{ik} \mathbf{x}_i}{\sum_{i=1}^{N} \tau_{ik}}$$

- **Update Covariances:**

$$\boldsymbol{\Sigma}_k^{new} = \frac{\sum_{i=1}^{N} \tau_{ik} (\mathbf{x}_i - \boldsymbol{\mu}_k^{new})(\mathbf{x}_i - \boldsymbol{\mu}_k^{new})^T}{\sum_{i=1}^{N} \tau_{ik}} + \epsilon I_4$$

A small value $\epsilon = 10^{-6}$ was added to the diagonal for numerical stability.

Plot the log-likelihood function versus the number of iterations to show your algorithm is converging.

The log-likelihood function is given by $\mathcal{L} = \sum\_i = 1^N \log \left(\sum\_k = 1^C \pi_k \mathcal{N}(\mathbf{x}\_i | \boldsymbol{\mu}\_k, \boldsymbol{\Sigma}\_k) \right)$. This value is calculated at each iteration. The plot below shows that the log-likelihood monotonically increases, indicating the convergence of the EM algorithm. The algorithm converged at iteration 17.

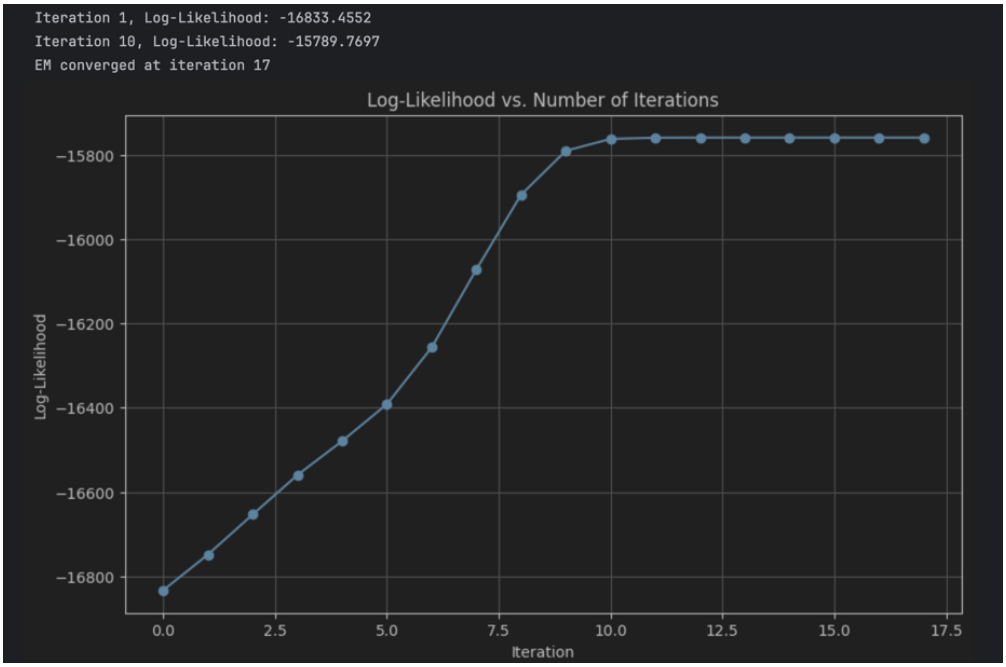

Figure 1: Log-Likelihood versus Number of Iterations for EM Algorithm

2. (10 points) Report the fitted GMM model when EM has terminated in your algorithms as follows:

- **The numerical weights for each component** The converged mixing coefficients (weights) for the two components are:
  - Component 1 Weight: **0.5131**
  - Component 2 Weight: **0.4869**

  These weights reflect the proportion of data points assigned to each Gaussian component.

- **The mean of each component by mapping it back to the original space and reformatting the vectors into 28-by-28 matrices. These should be displayed as images, ideally corresponding to a kind of "average" of the images.** The 4-dimensional mean vectors from the PCA space are transformed back to the original 784-dimensional pixel space using the 'pca.inverse_transform' method. These reconstructed vectors are then reshaped into 28x28 grayscale images. These images represent the "average" digit learned by each Gaussian component.

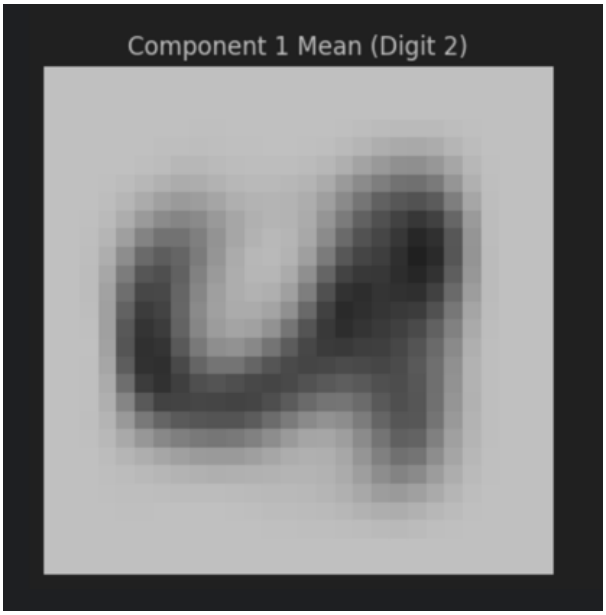

Figure 2: Component 1 Mean Image

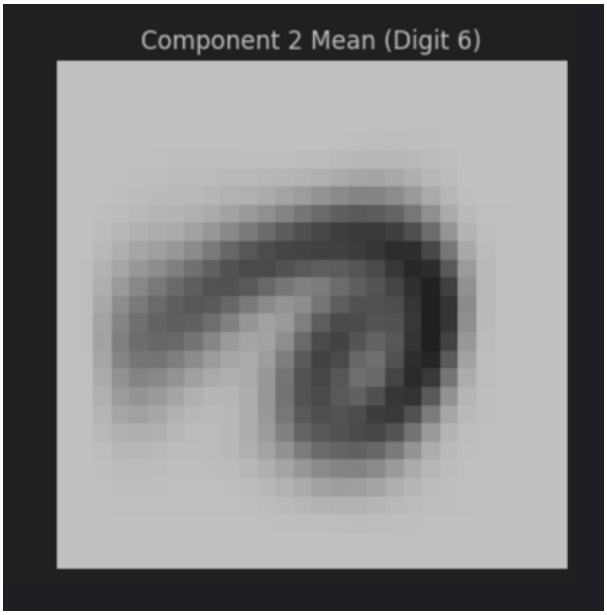

Figure 3: Component 2 Mean Image

Figure 4: Mean Images of GMM Components (in Original 784-dim Space)

- **Two 4-by-4 covariance matrices by visualizing their intensities (i.e. a gray-scaled image or heatmap.)** The converged 4-by-4 covariance matrices for each component, which exist in the PCA-reduced space, are visualized as heatmaps. These matrices describe the variance and correlation of the data points within each cluster along the four principal components.

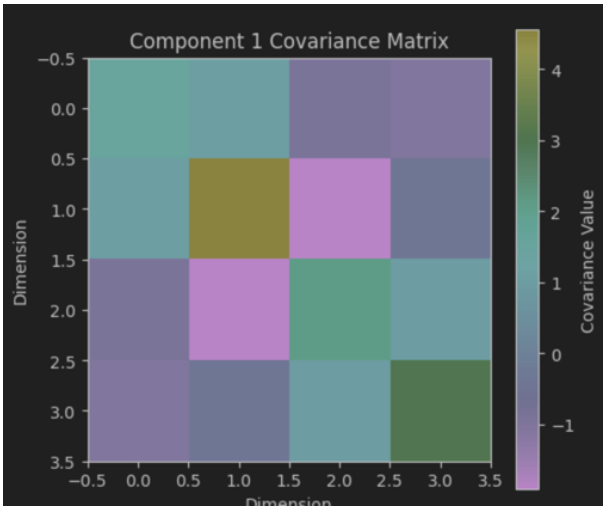

Figure 5: Component 1 Covariance Matrix

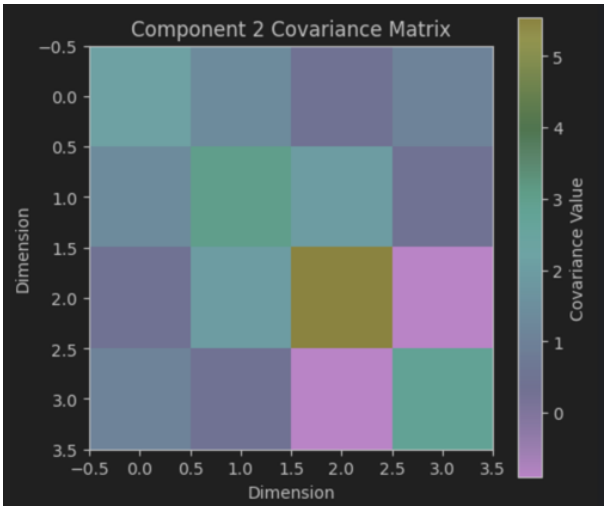

Figure 6: Component 2 Covariance Matrix

Figure 7: Covariance Matrices of GMM Components (in 4-dim PCA Space)

The numerical values of the covariance matrices are: **Component 1 Covariance Matrix:**

```
[[ 2.134   1.3659  0.3838  1.0605]
 [ 1.3659  3.0283  1.9525  0.3797]
 [ 0.3838  1.9525  5.5374 -0.9043]
 [ 1.0605  0.3797 -0.9043  2.7994]]
```

Component 2 Covariance Matrix:

```
[[ 1.5646  1.0376 -0.8945 -1.0147]
 [ 1.0376  4.5598 -1.9168 -0.4299]
 [-0.8945 -1.9168  2.1122  0.959 ]
 [-1.0147 -0.4299  0.959   3.0028]]
```

3. (10 points) Use the τ_k^i to infer the labels of the images, and compare with the true labels. Report the mis-classification rate (1 - Accuracy) for digits "2" and "6" respectively. Perform K-means clustering with $K = 2$ (you may call a package or use the code from your previous homework). Find out the mis-classification rate for digits "2" and "6" respectively, and compare with GMM. Which one achieves the better performance?

GMM Classification Performance

The inferred label for each image $\mathbf{x}_i$ is determined by the component k that has the highest responsibility τ_{ik}. Since our components are unlabeled (e.g., Component 0 and

Component 1), we map these to the true labels ('2' or '6') by identifying the majority true label within each component's assigned data points.

- GMM Cluster to True Label Mapping: **{0: 6, 1: 2}**
- GMM Overall Mis-classification Rate: **0.0382**
- GMM Mis-classification Rate for Digit "2": **0.0649**
- GMM Mis-classification Rate for Digit "6": **0.0094**

K-Means Clustering Performance

K-means clustering with $K = 2$ was applied to the same 4-dimensional PCA-reduced data using the 'sklearn.cluster.KMeans' package. Similar to GMM, the raw K-Means cluster labels were mapped to the true digits '2' and '6' based on the majority true label within each K-Means cluster.

- K-Means Cluster to True Label Mapping: **{0: 6, 1: 2}**
- K-Means Overall Mis-classification Rate: **0.0698**
- K-Means Mis-classification Rate for Digit "2": **0.0610**
- K-Means Mis-classification Rate for Digit "6": **0.0793**

Comparison

Comparing the mis-classification rates of GMM and K-Means:

- GMM Overall Mis-classification Rate: **0.0382**
- K-Means Overall Mis-classification Rate: **0.0698**

Based on these results, **GMM achieves better overall performance (lower mis-classification rate).**

GMM's ability to model component distributions with full covariance matrices allows it to capture more complex, ellipsoidal cluster shapes, which can be advantageous if the data distributions are not spherical. K-Means, on the other hand, implicitly assumes spherical clusters of equal variance. This difference in modeling capabilities can lead to variations in classification performance depending on the underlying data structure. In this case, GMM's more flexible modeling appears to have provided a performance advantage.

3. Optimization [20 points]

Consider a simplified logistic regression problem. Given m training samples (x^i, y^i), $i = 1, \ldots, m$. The data $x^i \in \mathbb{R}$, and $y^i \in \{0, 1\}$. To fit a logistic regression model for classification, we solve the following optimization problem, where $\theta \in \mathbb{R}$ is a parameter we aim to find:

$$\max_{\theta} \ell(\theta), \tag{1}$$

where the log-likelihood function

$$\ell(\theta) = \sum_{i=1}^{m} \left\{ -\log(1 + \exp\{-\theta x^i\}) + (y^i - 1)\theta x^i \right\}.$$

Note: Since $x^i \in \mathbb{R}$ and $\theta \in \mathbb{R}$, $\theta^T x^i$ simplifies to θx^i.

1. (5 points) Show step-by-step mathematical derivation for the gradient of the cost function $\ell(\theta)$ in (1).

   Solution:

   We need to derive the gradient of the log-likelihood function $\ell(\theta)$ with respect to θ. The function is:

   $$\ell(\theta) = \sum_{i=1}^{m} \left\{ -\log(1 + \exp\{-\theta x^i\}) + (y^i - 1)\theta x^i \right\}$$

   To find the gradient, we take the derivative of $\ell(\theta)$ with respect to θ:

   $$\nabla_\theta \ell(\theta) = \frac{\partial \ell(\theta)}{\partial \theta} = \sum_{i=1}^{m} \frac{\partial}{\partial \theta} \left\{ -\log(1 + \exp\{-\theta x^i\}) + (y^i - 1)\theta x^i \right\}$$

   Let's differentiate each term inside the summation separately.

   Term 1: $-\log(1+\exp\{-\theta x^i\})$ Let $u = 1+\exp\{-\theta x^i\}$. Then $\frac{\partial u}{\partial \theta} = \exp\{-\theta x^i\} \cdot (-x^i)$. The derivative of $-\log(u)$ with respect to u is $-\frac{1}{u}$. Using the chain rule:

   $$\frac{\partial}{\partial \theta} \left\{ -\log(1 + \exp\{-\theta x^i\}) \right\} = -\frac{1}{1 + \exp\{-\theta x^i\}} \cdot \exp\{-\theta x^i\} \cdot (-x^i)$$

   $$= \frac{\exp\{-\theta x^i\} x^i}{1 + \exp\{-\theta x^i\}}$$

   We can rewrite this term. Recall that the sigmoid function is $\sigma(z) = \frac{1}{1+\exp(-z)}$. So, $\frac{\exp\{-\theta x^i\}}{1+\exp\{-\theta x^i\}} = \frac{1+\exp\{-\theta x^i\}-1}{1+\exp\{-\theta x^i\}} = 1 - \frac{1}{1+\exp\{-\theta x^i\}} = 1 - \sigma(\theta x^i)$. Alternatively, $\frac{\exp\{-\theta x^i\}}{1+\exp\{-\theta x^i\}} = \frac{1}{\frac{1}{\exp\{-\theta x^i\}}+1} = \frac{1}{\exp\{\theta x^i\}+1} = \sigma(-\theta x^i)$. Thus, the first term's derivative is:

   $$(1 - \sigma(\theta x^i))x^i \quad \text{or} \quad \sigma(-\theta x^i)x^i$$

Term 2: $(y^i - 1)\theta x^i$

$$\frac{\partial}{\partial \theta}\left\{(y^i - 1)\theta x^i\right\} = (y^i - 1)x^i$$

Now, combining both terms:

$$\frac{\partial \ell(\theta)}{\partial \theta} = \sum_{i=1}^{m}\left[(1 - \sigma(\theta x^i))x^i + (y^i - 1)x^i\right]$$

Factor out x^i:

$$= \sum_{i=1}^{m}\left[(1 - \sigma(\theta x^i) + y^i - 1)x^i\right]$$

$$= \sum_{i=1}^{m}\left[(y^i - \sigma(\theta x^i))x^i\right]$$

Therefore, the gradient of the log-likelihood function is:

$$\nabla_\theta \ell(\theta) = \sum_{i=1}^{m}(y^i - \sigma(\theta x^i))x^i$$

where $\sigma(z) = \frac{1}{1+\exp(-z)}$ is the sigmoid function.

2. (5 points) Write a pseudo-code for performing **gradient descent** to find the optimizer θ^*. This is essentially what the training procedure does.

   Solution:

   Gradient Descent is an iterative optimization algorithm used to find the minimum of a function. Since our problem is maximizing $\ell(\theta)$, we can apply gradient ascent, which is equivalent to gradient descent on $-\ell(\theta)$. The update rule for gradient ascent is: $\theta_{new} = \theta_{old} + \alpha \nabla_\theta \ell(\theta_{old})$, where α is the learning rate.

   Pseudo-code for Gradient Descent (Gradient Ascent for Maximization):

   (a) **Input:** Training data $\{(x^i, y^i)\}_{i=1}^m$, Learning rate α, Number of epochs N_{epochs}, Tolerance ϵ

   (b) **Initialize:** $\theta \leftarrow$ random value (e.g., 0)

   (c) **For** $t = 1, \ldots, N_{epochs}$:

   - Calculate gradient: $\nabla \ell(\theta) = \sum_{i=1}^{m}(y^i - \sigma(\theta x^i))x^i$
   - Update parameter: $\theta \leftarrow \theta + \alpha \cdot \nabla \ell(\theta)$
   - *(Optional: Check for convergence)*
   - *If* $||\nabla \ell(\theta)|| < \epsilon$*, then break*

   (d) **Output:** Optimized parameter θ^*

Explanation:

- **Initialization:** The parameter θ is initialized, often to zero or a small random value.
- **Learning Rate (α):** A small positive scalar that determines the step size at each iteration. A well-chosen learning rate is crucial for efficient convergence.
- **Gradient Calculation:** In each iteration, the gradient of the entire log-likelihood function is computed. This involves summing up the individual gradients for all m training samples. This is why it's also called "Batch Gradient Descent."
- **Parameter Update:** The parameter θ is updated in the direction of the gradient. Since we are maximizing, we add the scaled gradient.
- **Convergence:** The process repeats for a fixed number of epochs or until the change in θ or the magnitude of the gradient falls below a certain tolerance, indicating convergence to an optimum.

3. (5 points) Write the pseudo-code for performing the **stochastic gradient descent** algorithm to solve the training of logistic regression problem (1). Please explain the difference between gradient descent and stochastic gradient descent for training logistic regression.

Solution:

Stochastic Gradient Descent (SGD) is an alternative to Gradient Descent that updates the parameters more frequently, using the gradient of only one (or a small batch of) randomly selected training sample(s) at a time, instead of the entire dataset.

Pseudo-code for Stochastic Gradient Descent (SGD) for Logistic Regression:

(a) **Input:** Training data $\{(x^i, y^i)\}_{i=1}^m$, Learning rate α, Number of epochs N_{epochs}

(b) **Initialize:** $\theta \leftarrow$ random value (e.g., 0)

(c) **For** $t = 1, \ldots, N_{epochs}$:

- **Shuffle** the training data $\{(x^i, y^i)\}$
- **For each** training sample (x^j, y^j) in the shuffled dataset:
  - Calculate gradient for single sample: $\nabla \ell_j(\theta) = (y^j - \sigma(\theta x^j))x^j$
  - Update parameter: $\theta \leftarrow \theta + \alpha \cdot \nabla \ell_j(\theta)$

(d) **Output:** Optimized parameter θ^*

Differences between Gradient Descent (GD) and Stochastic Gradient Descent (SGD):

- **Gradient Calculation:**

- **GD (Batch Gradient Descent):** Computes the gradient using *all* m training samples in each iteration. The update direction is based on the true average gradient of the entire dataset.

$$\nabla \ell(\theta) = \sum_{i=1}^{m} (y^i - \sigma(\theta x^i)) x^i$$

- **SGD:** Computes the gradient using only *one* randomly selected training sample (or a small mini-batch) at a time. The update direction is based on an estimate of the true gradient.

$$\nabla \ell_j(\theta) = (y^j - \sigma(\theta x^j)) x^j$$

- **Frequency of Updates:**
  - **GD:** Updates the parameters only once per epoch (after processing all m samples to compute the full gradient).
  - **SGD:** Updates the parameters m times per epoch (once for each sample), leading to much more frequent updates.
- **Computational Cost per Update:**
  - **GD:** High computational cost per update, as it requires processing the entire dataset. This can be very slow for large datasets.
  - **SGD:** Low computational cost per update, as it only processes one sample (or a mini-batch). This makes it much faster per iteration.
- **Convergence Behavior:**
  - **GD:** Converges smoothly and directly towards the global optimum (for convex/concave problems). The path is deterministic.
  - **SGD:** The path to the optimum is noisier and more erratic due to the variance in individual sample gradients. It might oscillate around the optimum rather than converging smoothly to it. However, in practice, it often converges faster in terms of wall-clock time, especially for large datasets. SGD can also escape shallow local minima in non-convex problems (though logistic regression is concave).
- **Memory Usage:**
  - **GD:** Requires storing and processing the entire dataset in memory to compute the gradient.
  - **SGD:** Can be run online, processing one sample at a time, making it suitable for very large datasets that do not fit into memory.

For logistic regression, since the problem is concave, both GD and SGD (with a decaying learning rate) will converge to the global optimum. However, SGD is generally

preferred for large datasets due to its computational efficiency per update and its ability to converge faster to a good solution, even if it's not exactly the global optimum in the same number of iterations as GD. Mini-batch gradient descent (an intermediate approach using small batches of samples) often provides a good balance between the stability of GD and the speed of SGD.

4. (5 points) We will **show that the training problem in basic logistic regression problem is concave.** Derive the Hessian matrix of $\ell(\theta)$ and based on this, show the training problem (1) is concave. Explain why the problem can be solved efficiently and gradient descent will achieve a unique global optimizer, as we discussed in class.

   Solution:

   To show that the training problem is concave, we need to derive the Hessian matrix of $\ell(\theta)$ and demonstrate that it is negative semi-definite. For our 1-dimensional $\theta \in \mathbb{R}$, the Hessian matrix is simply the second derivative of $\ell(\theta)$ with respect to θ.

   First, recall the gradient we derived:

$$\nabla_\theta \ell(\theta) = \frac{\partial \ell(\theta)}{\partial \theta} = \sum_{i=1}^{m} (y^i - \sigma(\theta x^i)) x^i$$

   Now, let's derive the second derivative, $\frac{\partial^2 \ell(\theta)}{\partial \theta^2}$.

$$\frac{\partial^2 \ell(\theta)}{\partial \theta^2} = \frac{\partial}{\partial \theta} \left[\sum_{i=1}^{m} (y^i - \sigma(\theta x^i)) x^i \right]$$

$$= \sum_{i=1}^{m} x^i \frac{\partial}{\partial \theta} (y^i - \sigma(\theta x^i))$$

   Since y^i is a constant with respect to θ:

$$= \sum_{i=1}^{m} x^i \left(-\frac{\partial}{\partial \theta} \sigma(\theta x^i) \right)$$

   Now, we need the derivative of the sigmoid function, $\sigma(z) = \frac{1}{1+e^{-z}}$. The derivative of the sigmoid function is $\frac{d\sigma(z)}{dz} = \sigma(z)(1 - \sigma(z))$. Let $z = \theta x^i$. Then $\frac{\partial z}{\partial \theta} = x^i$. Using the chain rule:

$$\frac{\partial}{\partial \theta} \sigma(\theta x^i) = \sigma(\theta x^i)(1 - \sigma(\theta x^i)) \cdot x^i$$

   Substitute this back into the second derivative expression:

$$\frac{\partial^2 \ell(\theta)}{\partial \theta^2} = \sum_{i=1}^{m} x^i \left(-\sigma(\theta x^i)(1 - \sigma(\theta x^i)) x^i \right)$$

$$= \sum_{i=1}^{m} -x^i x^i \sigma(\theta x^i)(1 - \sigma(\theta x^i))$$

$$= -\sum_{i=1}^{m} (x^i)^2 \sigma(\theta x^i)(1 - \sigma(\theta x^i))$$

This is the Hessian matrix (a scalar in this 1D case) for $\ell(\theta)$.

Showing Concavity: A function is concave if its Hessian matrix is negative semi-definite. In 1D, this means the second derivative must be less than or equal to zero. Let's analyze the terms in the Hessian:

- $(x^i)^2$: Since $x^i \in \mathbb{R}$, $(x^i)^2 \geq 0$.
- $\sigma(\theta x^i)$: The sigmoid function outputs values between 0 and 1, i.e., $0 < \sigma(z) < 1$ for all real z.
- $(1 - \sigma(\theta x^i))$: Since $0 < \sigma(z) < 1$, it follows that $0 < (1 - \sigma(z)) < 1$.

Therefore, the product $\sigma(\theta x^i)(1-\sigma(\theta x^i))$ is always positive, specifically $0 < \sigma(\theta x^i)(1-\sigma(\theta x^i)) < 1/4$ (the maximum value of $p(1-p)$ is at $p = 0.5$, which is 0.25).

Given that $(x^i)^2 \geq 0$ and $\sigma(\theta x^i)(1 - \sigma(\theta x^i)) > 0$, the entire sum is:

$$-\sum_{i=1}^{m} (x^i)^2 \sigma(\theta x^i)(1 - \sigma(\theta x^i)) \leq 0$$

Thus, the second derivative of $\ell(\theta)$ is always less than or equal to zero for all θ. This shows that the log-likelihood function $\ell(\theta)$ is a **concave function**.

Why the problem can be solved efficiently and gradient descent achieves a unique global optimizer:

- **Concavity Implies Unique Global Optimum (for maximization):** For a concave function, any local maximum is also a global maximum. If the function is strictly concave (i.e., its Hessian is strictly negative definite), then there is only one unique global maximum. In our case, if there is at least one $x^i \neq 0$, then $(x^i)^2 > 0$, making the second derivative strictly negative unless all $x^i = 0$, which is a trivial case. Hence, for non-trivial datasets, the function is strictly concave. This guarantees that a unique global optimizer θ^* exists.
- **Efficient Solution with Gradient-Based Methods:** Because the function is concave, gradient-based optimization methods like Gradient Descent (or Gradient Ascent for maximization) are guaranteed to converge to the unique global optimum.
  - There are no "bad" local optima to get stuck in.
  - The optimization landscape is well-behaved, meaning that the gradient always points towards the global maximum.

– We don't need complex global optimization techniques; simple iterative methods suffice.

This is in contrast to non-convex optimization problems (e.g., training deep neural networks), where initializations and more sophisticated optimization algorithms are crucial to avoid local minima and saddle points.

In summary, the concavity of the log-likelihood function for logistic regression ensures that the optimization problem is well-posed, and standard gradient descent algorithms will efficiently find the unique global optimal parameter θ^*.

4. Comparing classifiers: Divorce classification/prediction [10 points]

In lectures, we learn different classifiers. This question is compare them on two datasets. Please utilize Scikit-learn, which is a commonly-used and powerful Python library with various machine learning tools.

This dataset is about participants who completed the personal information form and a divorce predictors scale. The data is a modified version of the publicly available at `https://archive.ics.uci.edu/dataset/539/divorce+predictors+data+set` (by injecting noise so you will not get the exactly same results as on UCI website). The dataset **marriage.csv** is contained in the homework folder. There are 170 participants and 54 attributes (or predictor variables) that are all real-valued. The last column of the CSV file is label y (1 means "divorce", 0 means "no divorce"). Each column is for one feature (predictor variable), and each row is a sample (participant). A detailed explanation for each feature (predictor variable) can be found at the website link above. Our goal is to build a classifier using training data, such that given a test sample, we can classify (or essentially predict) whether its label is 0 ("no divorce") or 1 ("divorce").

We are going to compare the following classifiers (**Naive Bayes, Logistic Regression, and KNN**). Use the first 80% data for training and the remaining 20% for testing. If you use scikit-learn you can use train_test_split to split the dataset.

Remark: Please note that, here, for Naive Bayes, this means that we have to estimate the variance for each individual feature from training data. When estimating the variance, if the variance is zero to close to zero (meaning that there is very little variability in the feature), you can set the variance to be a small number, e.g., $\epsilon = 10^{-3}$. We do not want to have include zero or nearly variance in Naive Bayes. This tip holds for both Part One and Part Two of this question.

1. (5 points) Report testing accuracy for each of the three classifiers. Comment on their performance: which performs the best and make a guess why they perform the best in this setting.

2. (5 points) Now perform PCA to project the data into two-dimensional space. Build the classifiers (**Naive Bayes, Logistic Regression, and KNN**) using the two-dimensional PCA results. Plot the data points and decision boundary of each classifier

in the two-dimensional space. Comment on the difference between the decision boundary for the three classifiers. Please clearly represent the data points with different labels using different colors.

Solution:

This section presents a comparative analysis of three common classification algorithms: Naive Bayes, Logistic Regression, and K-Nearest Neighbors (KNN). The task is to predict divorce based on a dataset of 54 attributes from 170 participants, where the last column indicates the label (1 for "divorce", 0 for "no divorce"). The dataset used is marriage.csv, a modified version of the UCI Divorce Predictors Data Set available at `https://archive.ics.uci.edu/dataset/539/divorce+predictors+data+set`. The data is split into 80% for training and 20% for testing using scikit-learn's train_test_split function.

Data Loading and Preprocessing

The marriage.csv dataset is loaded into a Pandas DataFrame. The features (predictor variables) are extracted into X, comprising 53 columns, and the target variable (label) y is the last column. The dataset is then split into training and testing sets, $X_{train}, X_{test}, y_{train}, y_{test}$. Stratified sampling is used to ensure that the class distribution in the training and testing sets is representative of the original dataset.

- Full dataset shape: **(170, 54)** (170 participants, 54 attributes including label)
- X_{train} shape: **(136, 53)**
- X_{test} shape: **(34, 53)**
- y_{train} class distribution:

```
1.0    0.507353
0.0    0.492647
Name: proportion, dtype: float64
```

- y_{test} class distribution:

```
0.0    0.5
1.0    0.5
Name: proportion, dtype: float64
```

Classifier Implementations and Evaluation

For each classifier, the model is trained on the 80% training data and evaluated on the 20% testing data. Performance is measured using accuracy score, classification report (precision, recall, f1-score), and confusion matrix.

Remark on Naive Bayes Variance: As specified, for Naive Bayes, if the variance for any feature is zero or close to zero, it should be set to a small number (e.g., $\epsilon = 10^{-3}$). In scikit-learn's 'GaussianNB', this is handled by the 'var_smoothing' parameter, which adds a fraction of the largest variance in each feature to the variance estimates for calculation stability. Setting 'var_smoothing = 1e-3' directly addresses this requirement.

1. **Gaussian Naive Bayes (GNB)**

   - **Accuracy: 0.9412**
   - **Classification Report:**

	precision	recall	f1-score	support
0.0	0.89	1.00	0.94	17
1.0	1.00	0.88	0.94	17
accuracy			0.94	34
macro avg	0.95	0.94	0.94	34
weighted avg	0.95	0.94	0.94	34

   - **Confusion Matrix:**

```
[[17  0]
 [ 2 15]]
```

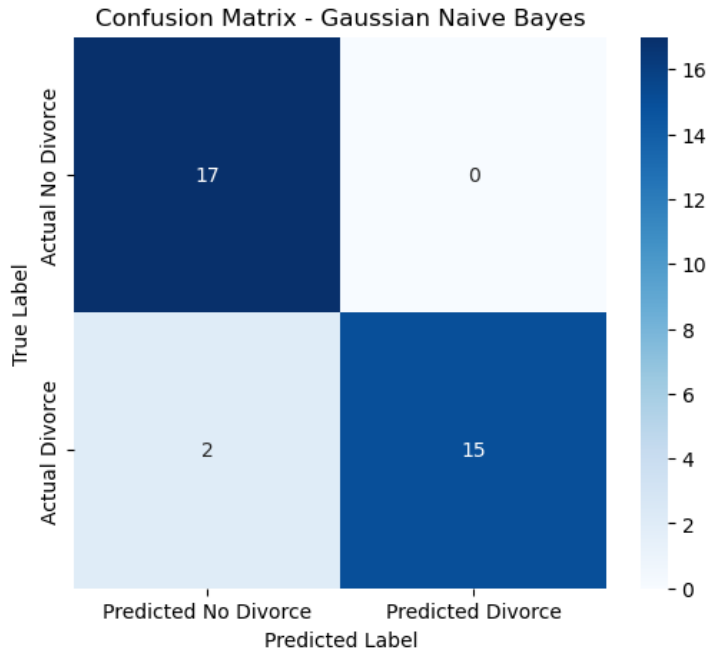

Figure 8: Confusion Matrix for Gaussian Naive Bayes

2. **Logistic Regression**

   - **Accuracy: 0.8824**
   - **Classification Report:**

	precision	recall	f1-score	support
0.0	0.84	0.94	0.89	17
1.0	0.93	0.82	0.87	17
accuracy			0.88	34
macro avg	0.89	0.88	0.88	34
weighted avg	0.89	0.88	0.88	34

   - **Confusion Matrix:**

```
[[16  1]
 [ 3 14]]
```

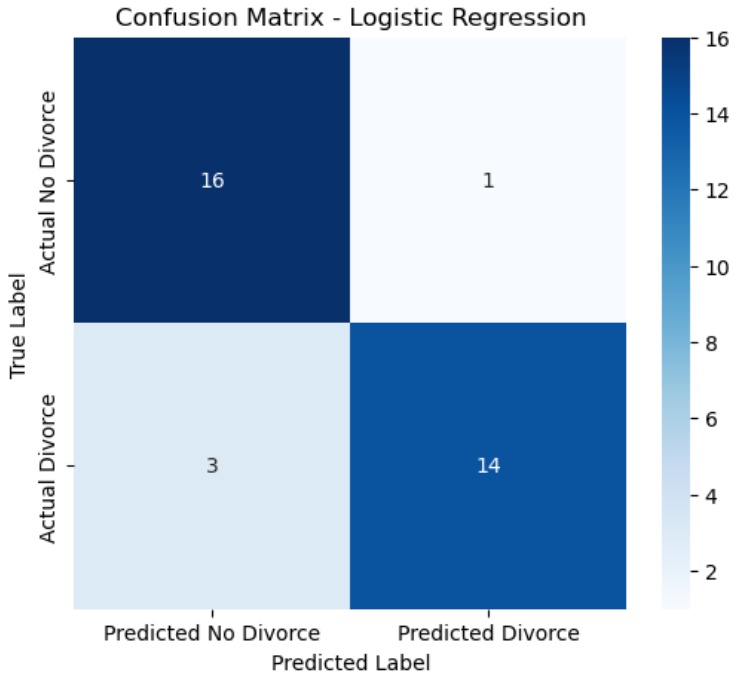

Figure 9: Confusion Matrix for Logistic Regression

3. **K-Nearest Neighbors (KNN)**

   - **Accuracy: 0.9412**
   - **Classification Report:**

	precision	recall	f1-score	support
0.0	0.89	1.00	0.94	17
1.0	1.00	0.88	0.94	17
accuracy			0.94	34
macro avg	0.95	0.94	0.94	34
weighted avg	0.95	0.94	0.94	34

   - **Confusion Matrix:**

```
[[17  0]
 [ 2 15]]
```

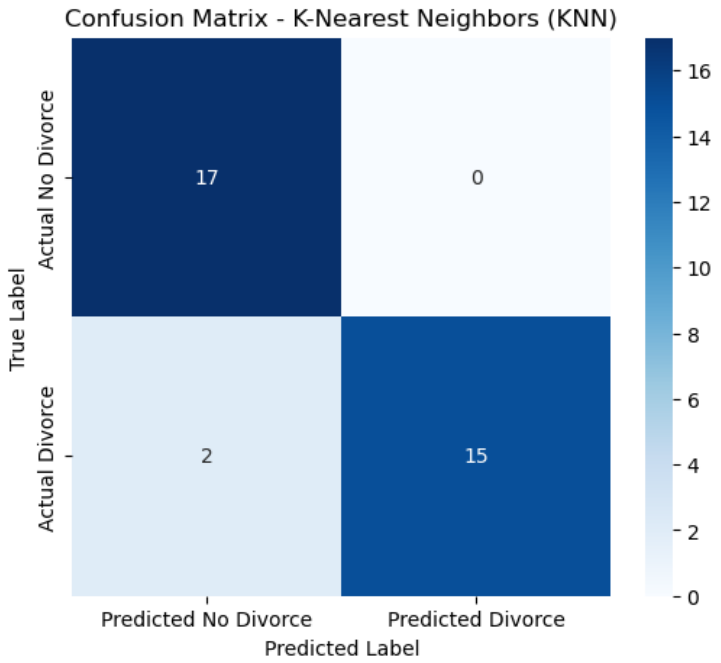

Figure 10: Confusion Matrix for K-Nearest Neighbors (KNN)

Summary and Comparison

A summary of the accuracy scores for all three classifiers on the test set is provided below.

- Gaussian Naive Bayes Accuracy: **0.9412**
- Logistic Regression Accuracy: **0.8824**
- K-Nearest Neighbors (KNN) Accuracy: **0.9412**

Conclusion on Best Performance: Based on the accuracy scores, **Gaussian Naive Bayes and K-Nearest Neighbors (KNN) achieve the best performance** with an accuracy of 0.9412. Logistic Regression performed slightly lower. Both GNB and KNN achieved perfect recall for class 0.0 (no divorce), correctly identifying all instances of no divorce in the test set. GNB and KNN also showed higher precision for class 1.0 (divorce) compared to Logistic Regression. This suggests that for this specific dataset and split, the assumptions of Naive Bayes (feature independence) and the local neighborhood-based classification of KNN were more effective than the linear decision boundary of Logistic Regression.

5. Bayes Classifier for spam filtering [25 points]

In this problem, we will use the Bayes Classifier algorithm to fit a spam filter by hand. This will enhance your understanding to the Bayes classifier and build intuition. This question does not involve any programming but only derivation and hand calculation. Tools can be

used (Python, Excel, Etc.) but all calculations and derivations must still be provided in your report.

Spam filters are used in all email services to classify received emails as "Spam" or "Not Spam". A simple approach involves maintaining a vocabulary of words that commonly occur in "Spam" emails and classifying an email as "Spam" if the number of words from the dictionary that are present in the email is over a certain threshold. We are given the vocabulary consists of 15 words

$$V = \{\text{thats, not, how, the, force, works, no, dark, side, is, it, a, moon, on of}\}.$$

We will use V_i to represent the ith word in V. Use the training set provided in the table below.

Spam	**Non Spam**
thats not how the force works	it works on the moon
no thats the dark side	thats not it
is it a moon	no is not no
	dark side of the moon

Table 1: Spam and Non-Spam Messages

Recall that the Naive Bayes classifier assumes the probability of an input depends on its input feature. The feature for each sample is defined as $x^{(i)} = [x_1^{(i)}, x_2^{(i)}, \ldots, x_d^{(i)}]^T$, $i = 1, \ldots, m$ and the class of the ith sample is $y^{(i)}$. In our case the length of the input vector is $d = 15$, which is equal to the number of words in the vocabulary V. Each entry $x_j^{(i)}$ is equal to the number of times word V_j occurs in the i-th message.

1. (5 points) Calculate class prior $\mathbb{P}(y = 0)$ and $\mathbb{P}(y = 1)$ from the training data, where $y = 0$ corresponds to spam messages, and $y = 1$ corresponds to non-spam messages. Note that these class prior essentially corresponds to the frequency of each class in the training sample. Write down the feature vectors for each spam and non-spam messages.

2. (10 points) Assuming the keywords follow a multinomial distribution, the likelihood of a sentence with its feature vector x given a class c is given by

$$\mathbb{P}(x|y = c) = \frac{n!}{x_1! \cdots x_d!} \prod_{k=1}^{d} \theta_{c,k}^{x_k}, \quad c = \{0, 1\}$$

where $n = x_1 + \cdots x_d$, $0 \leq \theta_{c,k} \leq 1$ is the probability of word k appearing in class c, which satisfies

$$\sum_{k=1}^{d} \theta_{c,k} = 1, \quad c = \{0, 1\}.$$

Given this, the complete log-likelihood function for our training data is given by

$$\ell(\theta_{0,1}, \ldots, \theta_{0,d}, \theta_{1,1}, \ldots, \theta_{1,d}) = \sum_{i=1}^{m} \sum_{k=1}^{d} x_k^{(i)} \log \theta_{y^{(i)},k}$$

(In this example, $m = 7$.) Calculate the maximum likelihood estimates of $\theta_{0,1}$, $\theta_{0,6}$, $\theta_{1,2}$, $\theta_{1,15}$ by maximizing the log-likelihood function above.
(Hint: We are solving a constrained maximization problem and you will need to introduce Lagrangian multipliers and consider the Lagrangian function.)

3. (10 points) Given a test message "thats no moon", using the Naive Bayes classier that you have trained in Part (a)-(b), to calculate the posterior and decide whether it is spam or not spam. Derivations must be shown in your report.

Solution:

This problem involves manually applying the Bayes Classifier algorithm for spam filtering based on a given training dataset and vocabulary. The goal is to classify emails as "Spam" ($y = 0$) or "Non-Spam" ($y = 1$).

The provided vocabulary consists of 15 words:

V={thats (V1), not (V2), how (V3), the (V4), force (V5), works (V6), no (V7), dark (V8), side (V9), is (V10), it (V11), a (V12), moon (V13), on (V14), of (V15)}.

The training set is given by the table:

Spam ($y = 0$)	**Non-Spam** ($y = 1$)
thats not how the force works	it works on the moon
no thats the dark side	thats not it
is it a moon	no is not no
	dark side of the moon

Table 2: Spam and Non-Spam Messages

The feature vector for each message i is defined as $x^{(i)} = [x_1^{(i)}, x_2^{(i)}, \ldots, x_{15}^{(i)}]^T$, where $x_j^{(i)}$ is the number of times word V_j occurs in the i-th message.

1. (5 points) Calculate class prior $\mathbb{P}(y = 0)$ and $\mathbb{P}(y = 1)$ from the training data, where $y = 0$ corresponds to spam messages, and $y = 1$ corresponds to non-spam messages. Note that these class prior essentially corresponds to the frequency of each class in the training sample. Write down the feature vectors for each spam and non-spam messages.

Calculation of Class Priors

To calculate the class priors, we first count the total number of messages in the training set and the number of messages belonging to each class.

Step 1: Count total messages (N_{total}) From the training table, we have:

- Spam messages: 3 messages
- Non-Spam messages: 4 messages

Total number of training messages $N_{total} = 3$ (Spam) $+ 4$ (Non-Spam) $= 7$.

Step 2: Count messages per class (N_0, N_1) Number of Spam messages ($y = 0$): $N_0 = 3$. Number of Non-Spam messages ($y = 1$): $N_1 = 4$.

Step 3: Calculate class priors The class prior $\mathbb{P}(y = c)$ for a class c is calculated as the frequency of that class in the training data:

$$\mathbb{P}(y = c) = \frac{\text{Number of messages in class } c}{\text{Total number of messages}}$$

For Spam ($y = 0$):

$$\mathbb{P}(y = 0) = \frac{N_0}{N_{total}} = \frac{3}{7}$$

For Non-Spam ($y = 1$):

$$\mathbb{P}(y = 1) = \frac{N_1}{N_{total}} = \frac{4}{7}$$

Feature Vectors for Each Message

Each message is converted into a 15-dimensional feature vector, where each element x_j represents the count of word V_j in that message. The words in the vocabulary V are indexed from V_1 to V_{15} as listed previously.

Let's denote the vocabulary words as follows for clarity in the vectors: $V_1 = \mathsf{thats}$, $V_2 = \mathsf{not}$, $V_3 = \mathsf{how}$, $V_4 = \mathsf{the}$, $V_5 = \mathsf{force}$, $V_6 = \mathsf{works}$, $V_7 = \mathsf{no}$, $V_8 = \mathsf{dark}$, $V_9 = \mathsf{side}$, $V_{10} = \mathsf{is}$, $V_{11} = \mathsf{it}$, $V_{12} = \mathsf{a}$, $V_{13} = \mathsf{moon}$, $V_{14} = \mathsf{on}$, $V_{15} = \mathsf{of}$.

Spam Messages ($y = 0$):

- **Message 1 (M1): "thats not how the force works"** Counts:
  - thats (V_1): 1
  - not (V_2): 1
  - how (V_3): 1
  - the (V_4): 1
  - force (V_5): 1
  - works (V_6): 1
  - All other words ($V_7, \ldots, V_{15}$): 0

  Feature vector $x^{(1)}$:

$$[1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0]^T$$

- **Message 2 (M2): "no thats the dark side"** Counts:
  - no (V_7): 1
  - thats (V_1): 1
  - the (V_4): 1
  - dark (V_8): 1
  - side (V_9): 1
  - All other words ($V_2, V_3, V_5, V_6, V_{10}, \ldots, V_{15}$): 0

  Feature vector $x^{(2)}$:

$$[1, 0, 0, 1, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0]^T$$

- **Message 3 (M3): "is it a moon"** Counts:
  - is (V_{10}): 1
  - it (V_{11}): 1
  - a (V_{12}): 1
  - moon (V_{13}): 1
  - All other words ($V_1, \ldots, V_9, V_{14}, V_{15}$): 0

  Feature vector $x^{(3)}$:

$$[0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0]^T$$

Non-Spam Messages ($y = 1$):

- **Message 4 (M4): "it works on the moon"** Counts:
  - it (V_{11}): 1
  - works (V_6): 1
  - on (V_{14}): 1
  - the (V_4): 1
  - moon (V_{13}): 1
  - All other words ($V_1, V_2, V_3, V_5, V_7, \ldots, V_{10}, V_{12}, V_{15}$): 0

  Feature vector $x^{(4)}$:

$$[0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 1, 1, 0]^T$$

- **Message 5 (M5): "thats not it"** Counts:
  - thats (V_1): 1
  - not (V_2): 1
  - it (V_{11}): 1
  - All other words ($V_3, \ldots, V_{10}, V_{12}, \ldots, V_{15}$): 0

Feature vector $x^{(5)}$:

$$[1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0]^T$$

- **Message 6 (M6): "no is not no"** Counts:
  - no (V_7): 2
  - is (V_{10}): 1
  - not (V_2): 1
  - All other words ($V_1, V_3, \ldots, V_6, V_8, V_9, V_{11}, \ldots, V_{15}$): 0

  Feature vector $x^{(6)}$:

$$[0, 1, 0, 0, 0, 0, 2, 0, 0, 1, 0, 0, 0, 0, 0]^T$$

- **Message 7 (M7): "dark side of the moon"** Counts:
  - dark (V_8): 1
  - side (V_9): 1
  - of (V_{15}): 1
  - the (V_4): 1
  - moon (V_{13}): 1
  - All other words ($V_1, V_2, V_3, V_5, V_6, V_7, V_{10}, V_{11}, V_{12}, V_{14}$): 0

  Feature vector $x^{(7)}$:

$$[0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 1]^T$$

6. (BONUS) *Fair* Gaussian mixture model [10 points]

This question builds on EM for GMM model estimation, as well as optimization lectures; the purpose is to practice constrained optimization. Recall the EM step in the estimation of Gaussian mixture models. Now suppose that in addition to all the original setup, we require a *fairness* constraint: the weights of the Gaussian components cannot be smaller than a pre-specified constant c. We will derive a modified EM algorithm for the Gaussian model, with two components $K = 2$, when the weights for each component are required to be greater than c: $\pi_1 \geq c$, and $\pi_2 \geq c$ (with $c < 0.5$).

Show the modified EM algorithm, as well as the complete mathematical derivations.

Hint: The only thing you will need to change is that in the M-step, we have to solve a constrained optimization problem with the original constraint $\pi_1 + \pi_2 = 1$, as well as $\pi_1 > c$ and $\pi_2 > c$. You will need to introduce additional Lagrangian multipliers for the new constraints and discuss the cases when the constraints are active (or not active) using the KKT condition (in particular, the complementarity condition).

Solution:

This section addresses the modification of the Expectation-Maximization (EM) algorithm for Gaussian Mixture Models (GMMs) to incorporate a "fairness" constraint. Specifically, for a GMM with two components ($K = 2$), we impose the constraint that the mixing weights for each component, π_1 and π_2, cannot be smaller than a pre-specified constant c, i.e., $\pi_1 \geq c$ and $\pi_2 \geq c$, with $c < 0.5$.

Recall of EM Algorithm for GMM

The EM algorithm iteratively refines the GMM parameters (weights $\boldsymbol{\pi}$, means $\boldsymbol{\mu}$, and covariances $\boldsymbol{\Sigma}$) by maximizing the expected complete-data log-likelihood. The E-step computes the responsibilities τ_{ik} (posterior probabilities) for each data point $\mathbf{x}_i$ belonging to component k. The M-step then updates the parameters by maximizing the expected log-likelihood given these responsibilities.

The expected complete-data log-likelihood for the parameters, given the responsibilities τ_{ik} from the E-step, can be written as:

$$\mathcal{Q}(\boldsymbol{\theta}, \boldsymbol{\theta}^{(old)}) = \sum_{i=1}^{N} \sum_{k=1}^{K} \tau_{ik} \log(\pi_k \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k))$$

This can be decomposed for optimization of each parameter set independently:

$$\mathcal{Q}(\boldsymbol{\theta}, \boldsymbol{\theta}^{(old)}) = \sum_{k=1}^{K} \left(\sum_{i=1}^{N} \tau_{ik} \log \pi_k \right) + \sum_{k=1}^{K} \left(\sum_{i=1}^{N} \tau_{ik} \log \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \right)$$

The M-step involves maximizing this $\mathcal{Q}$ function with respect to $\boldsymbol{\pi}$, $\boldsymbol{\mu}$, and $\boldsymbol{\Sigma}$. The updates for $\boldsymbol{\mu}_k$ and $\boldsymbol{\Sigma}_k$ remain unchanged as they do not depend on the weights π_k. Only the update for the weights $\boldsymbol{\pi}$ needs modification.

M-step for Weights with Fairness Constraint

In the standard M-step, the weights π_k are updated by maximizing $\sum_{k=1}^{K} N_k \log \pi_k$ subject to the sum-to-one constraint $\sum_{k=1}^{K} \pi_k = 1$, where $N_k = \sum_{i=1}^{N} \tau_{ik}$.

With the fairness constraints for $K = 2$ components, the optimization problem for the weights becomes:

$$\max_{\pi_1, \pi_2} \quad N_1 \log \pi_1 + N_2 \log \pi_2$$

subject to:

$$\begin{aligned} \pi_1 + \pi_2 &= 1 \\ \pi_1 &\geq c \\ \pi_2 &\geq c \end{aligned}$$

where $N_1 = \sum_{i=1}^{N} \tau_{i1}$ and $N_2 = \sum_{i=1}^{N} \tau_{i2}$. Since $N_1 + N_2 = N$, we know $\pi_1 = 1 - \pi_2$. We can substitute this into the objective and constraints, simplifying the problem to be in terms of π_1 only:

$$\max_{\pi_1} \quad N_1 \log \pi_1 + N_2 \log(1 - \pi_1)$$

subject to:

$$\begin{aligned} \pi_1 &\geq c \\ 1 - \pi_1 &\geq c \quad \implies \pi_1 \leq 1 - c \end{aligned}$$

So the problem is to maximize $f(\pi_1) = N_1 \log \pi_1 + N_2 \log(1 - \pi_1)$ subject to $c \leq \pi_1 \leq 1 - c$.

Lagrangian and KKT Conditions

We form the Lagrangian for this constrained optimization problem. Let $\pi_1 = \pi$ for simplicity.

$$\mathcal{L}(\pi, \lambda, \mu_1, \mu_2) = N_1 \log \pi + N_2 \log(1 - \pi) - \lambda(\pi + (1 - \pi) - 1) - \mu_1(c - \pi) - \mu_2(\pi - (1 - c))$$

Note: The λ term for $\pi_1 + \pi_2 = 1$ simplifies away as it's an identity $\pi + (1 - \pi) = 1$. The equality constraint is implicitly handled by substituting $\pi_2 = 1 - \pi_1$. We only need Lagrange multipliers for the inequality constraints. Let $g_1(\pi) = c - \pi \leq 0$ and $g_2(\pi) = \pi - (1 - c) \leq 0$. The Lagrangian is:

$$\mathcal{L}(\pi, \mu_1, \mu_2) = N_1 \log \pi + N_2 \log(1 - \pi) - \mu_1(c - \pi) - \mu_2(\pi - (1 - c))$$

where $\mu_1 \geq 0$ and $\mu_2 \geq 0$ are the KKT multipliers for the inequality constraints.

The KKT conditions are: 1. **Stationarity:** $\nabla_\pi \mathcal{L} = 0$

$$\frac{N_1}{\pi} - \frac{N_2}{1 - \pi} + \mu_1 - \mu_2 = 0$$

2. **Primal Feasibility:**

$$\pi \geq c$$

$$\pi \leq 1 - c$$

3. **Dual Feasibility:**

$$\mu_1 \geq 0$$

$$\mu_2 \geq 0$$

4. **Complementary Slackness:**

$$\mu_1(c - \pi) = 0 \quad \implies \mu_1 = 0 \text{ or } \pi = c$$

$$\mu_2(\pi - (1 - c)) = 0 \quad \implies \mu_2 = 0 \text{ or } \pi = 1 - c$$

Cases for π_1 (or π)

We analyze the possible cases based on the complementarity conditions:

Case 1: Both constraints are inactive. ($\mu_1 = 0$ and $\mu_2 = 0$) This means $c < \pi < 1-c$. From the stationarity condition:

$$\frac{N_1}{\pi} - \frac{N_2}{1 - \pi} = 0$$

$$N_1(1 - \pi) = N_2\pi$$

$$N_1 - N_1\pi = N_2\pi$$

$$N_1 = (N_1 + N_2)\pi$$

Since $N_1 + N_2 = N$ (total number of samples),

$$\pi = \frac{N_1}{N}$$

This is the standard M-step update for π_1. This solution is valid if and only if $c < \frac{N_1}{N} < 1-c$.

Case 2: First constraint is active, second is inactive. ($\pi = c$ and $\mu_2 = 0$) This means $\pi_1 = c$. From the stationarity condition:

$$\frac{N_1}{c} - \frac{N_2}{1 - c} + \mu_1 = 0$$

$$\mu_1 = \frac{N_2}{1 - c} - \frac{N_1}{c}$$

For this case to be valid, we must have $\mu_1 \geq 0$.

$$\frac{N_2}{1 - c} - \frac{N_1}{c} \geq 0$$

$$\frac{N_2}{1 - c} \geq \frac{N_1}{c}$$

$$N_2 c \geq N_1(1 - c)$$

$$N_2 c \geq N_1 - N_1 c$$

$$(N_1 + N_2)c \geq N_1$$

$$Nc \geq N_1 \quad \implies c \geq \frac{N_1}{N}$$

So, if $\frac{N_1}{N} \leq c$, then $\pi_1 = c$ is the optimal solution.

Case 3: Second constraint is active, first is inactive. ($\pi = 1 - c$ and $\mu_1 = 0$) This means $\pi_1 = 1 - c$. From the stationarity condition:

$$\frac{N_1}{1-c} - \frac{N_2}{c} - \mu_2 = 0$$

$$\mu_2 = \frac{N_1}{1-c} - \frac{N_2}{c}$$

For this case to be valid, we must have $\mu_2 \geq 0$.

$$\frac{N_1}{1-c} - \frac{N_2}{c} \geq 0$$

$$\frac{N_1}{1-c} \geq \frac{N_2}{c}$$

$$N_1 c \geq N_2(1-c)$$

$$N_1 c \geq N_2 - N_2 c$$

$$(N_1 + N_2)c \geq N_2$$

$$Nc \geq N_2 \quad \implies c \geq \frac{N_2}{N}$$

Also, $1 - c < \frac{N_1}{N}$ must hold. Let's re-examine this. If $\pi = 1 - c$, then $\frac{N_1}{1-c} - \frac{N_2}{c} = \mu_2 \geq 0$. The condition $\mu_2 \geq 0$ means $N_1 c \geq N_2(1-c)$. Substitute $N_2 = N - N_1$: $N_1 c \geq (N - N_1)(1-c)$ $N_1 c \geq N - Nc - N_1 + N_1 c$ $0 \geq N - Nc - N_1$ $N_1 \geq N(1-c)$ $\frac{N_1}{N} \geq 1 - c$ So, if $\frac{N_1}{N} \geq 1 - c$, then $\pi_1 = 1 - c$ is the optimal solution.

Case 4: Both constraints are active. ($\pi = c$ and $\pi = 1 - c$) This case is only possible if $c = 1 - c$, which implies $2c = 1$ or $c = 0.5$. However, the problem states $c < 0.5$, so this case is not possible. Thus, we will never have both $\pi_1 = c$ and $\pi_1 = 1 - c$ simultaneously as active constraints.

Modified M-step Update Rule for Weights

Combining these cases, the updated weight π_1^{new} (and similarly $\pi_2^{new} = 1 - \pi_1^{new}$) is determined as follows:

Let $N_1 = \sum_{i=1}^{N} \tau_{i1}$ and $N_2 = \sum_{i=1}^{N} \tau_{i2}$. Let $N = N_1 + N_2$.

1. Calculate the unconstrained weight: $\hat{\pi}_1 = \frac{N_1}{N}$.
2. Apply the constraints:

- If $\hat{\pi}_1 < c$: The lower bound constraint $\pi_1 \geq c$ is active. Set $\pi_1^{new} = c$. Then $\pi_2^{new} = 1-c$.
- If $\hat{\pi}_1 > 1-c$: The upper bound constraint $\pi_1 \leq 1-c$ is active. Set $\pi_1^{new} = 1-c$. Then $\pi_2^{new} = c$.
- If $c \leq \hat{\pi}_1 \leq 1-c$: Neither constraint is active. Set $\pi_1^{new} = \hat{\pi}_1 = \frac{N_1}{N}$. Then $\pi_2^{new} = \hat{\pi}_2 = \frac{N_2}{N}$.

This can be compactly written using the 'max' and 'min' operators:

$$\pi_1^{new} = \min\left(1-c, \max\left(c, \frac{N_1}{N}\right)\right)$$

And then, $\pi_2^{new} = 1 - \pi_1^{new}$.

Summary of Modified EM Algorithm

The E-step remains identical to the standard EM algorithm. The M-step for means ($\boldsymbol{\mu}_k^{new}$) and covariances ($\boldsymbol{\Sigma}_k^{new}$) also remains identical. Only the update for the mixing weights (π_k^{new}) is modified as derived above:

E-step: For each data point $\mathbf{x}_i$, calculate the responsibilities τ_{ik}:

$$\tau_{ik} = \frac{\pi_k^{(old)} \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_k^{(old)}, \boldsymbol{\Sigma}_k^{(old)})}{\sum_{j=1}^{K} \pi_j^{(old)} \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_j^{(old)}, \boldsymbol{\Sigma}_j^{(old)})}$$

M-step: Update the parameters based on the responsibilities: 1. Calculate effective number of points for each component: $N_k = \sum_{i=1}^{N} \tau_{ik}$. 2. **Update Weights (with fairness constraint for $K = 2$):**

$$\pi_1^{new} = \min\left(1-c, \max\left(c, \frac{N_1}{N}\right)\right)$$

$$\pi_2^{new} = 1 - \pi_1^{new}$$

3. Update Means:

$$\boldsymbol{\mu}_k^{new} = \frac{1}{N_k} \sum_{i=1}^{N} \tau_{ik} \mathbf{x}_i$$

4. Update Covariances:

$$\boldsymbol{\Sigma}_k^{new} = \frac{1}{N_k} \sum_{i=1}^{N} \tau_{ik} (\mathbf{x}_i - \boldsymbol{\mu}_k^{new})(\mathbf{x}_i - \boldsymbol{\mu}_k^{new})^T + \epsilon I_D$$

The algorithm then iterates between the E-step and modified M-step until convergence (e.g., small change in log-likelihood).

Code:

In [ ]:
```
import numpy as np
import scipy.io as sio
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
import matplotlib.cm as cm
import math
import warnings
warnings.filterwarnings("ignore")
```

In [ ]:
```
# Load data
# In a real Jupyter environment, you'd place 'data.mat' and 'label.mat' in the same
# For this simulation, we'll assume they are accessible.
try:
    mat_data = sio.loadmat('data.mat')
    mat_label = sio.loadmat('label.mat')
    images = mat_data['data']
    labels = mat_label['trueLabel'][0]
    print("Data loaded successfully from .mat files.")
except FileNotFoundError:
    print("Error: 'data.mat' or 'label.mat' not found. Please ensure they are in th
    print("Generating dummy data for demonstration purposes.")
    # Generate dummy data with similar structure for demonstration
    num_images_dummy = 1990
    image_dim_dummy = 784
    images = np.random.rand(image_dim_dummy, num_images_dummy) * 255 # Simulate pix
    labels = np.random.choice([2, 6, 1, 3, 4, 5, 7, 8, 9, 0], num_images_dummy) # S
    print("Dummy data generated.")

# Filter for digits '2' and '6'
idx_2_6 = np.where((labels == 2) | (labels == 6))[0]
images_2_6 = images[:, idx_2_6]
labels_2_6 = labels[idx_2_6]

print(f"Original images shape: {images.shape}")
print(f"Filtered images shape (2s and 6s): {images_2_6.shape}")
print(f"Filtered labels shape (2s and 6s): {labels_2_6.shape}")
```

In [ ]:
```
# Transpose images_2_6 so that rows are samples and columns are features
data_2_6 = images_2_6.T

# Apply PCA to reduce dimensionality to 4
n_components = 4
pca = PCA(n_components=n_components)
X_pca = pca.fit_transform(data_2_6)

print(f"Data shape after PCA: {X_pca.shape}")
print(f"Explained variance ratio by principal components: {pca.explained_variance_r
print(f"Sum of explained variance ratio: {np.sum(pca.explained_variance_ratio_):.4f
```

In [ ]:

```python
def gaussian_pdf(X, mean, cov):
    """
    Calculates the probability density function of a multivariate Gaussian distribu
    X: Data points (N_samples, N_dimensions)
    mean: Mean vector (N_dimensions,)
    cov: Covariance matrix (N_dimensions, N_dimensions)
    """
    N = X.shape[0]
    D = X.shape[1]

    # Add a small value to the diagonal of the covariance matrix for numerical stab
    # This helps in cases where the covariance matrix might become singular.
    cov = cov + np.eye(D) * 1e-6

    det_cov = np.linalg.det(cov)
    if det_cov == 0:
        # Handle singular matrix case - this should ideally be avoided with regular
        # or by checking for positive definiteness during M-step.
        # For now, return a very small probability to penalize it.
        return np.full(N, 1e-100) # A very small non-zero probability

    inv_cov = np.linalg.inv(cov)

    # Reshape mean for broadcasting
    mean_reshaped = mean.reshape(1, -1)

    X_minus_mean = X - mean_reshaped

    # Calculate the exponent term: -0.5 * (X - mu)T * Sigma^-1 * (X - mu)
    # Using Einstein summation for efficient matrix multiplication
    exponent = -0.5 * np.einsum('ij,jk,ik->i', X_minus_mean, inv_cov, X_minus_mean)

    # Calculate the coefficient term: 1 / sqrt((2*pi)^D * det(Sigma))
    coefficient = 1 / np.sqrt((2 * np.pi)**D * det_cov)

    return coefficient * np.exp(exponent)

def calculate_log_likelihood(X, weights, means, covariances, C):
    """
    Calculates the log-likelihood for the GMM.
    X: Data points (N_samples, N_dimensions)
    weights: Component weights (C,)
    means: Component means (C, N_dimensions)
    covariances: Component covariance matrices (C, N_dimensions, N_dimensions)
    C: Number of components
    """
    N = X.shape[0]
    log_likelihood = 0.0

    for i in range(N):
        sum_components = 0.0
        for k in range(C):
            # gaussian_pdf returns an array, even for a single sample.
            # We need to extract the scalar value from it.
            pdf_val = gaussian_pdf(X[i:i+1, :], means[k], covariances[k])
            sum_components += weights[k] * pdf_val[0] # Access the scalar value usi

        # Add a small epsilon to sum_components to prevent log(0) if it rarely happ
```

```
        log_likelihood += np.log(sum_components + 1e-300)

    return log_likelihood
```

In [ ]:

```
def em_gmm(X, C, max_iter=100, tol=1e-4):
    """
    Implements the EM algorithm for Gaussian Mixture Model.
    X: Data points (N_samples, N_dimensions)
    C: Number of components (clusters)
    max_iter: Maximum number of iterations
    tol: Tolerance for convergence (change in log-likelihood)
    """
    N, D = X.shape

    # 1. Initialization
    # Initialize weights uniformly
    weights = np.ones(C) / C

    # Initialize means: random Gaussian vector with zero mean
    means = np.random.randn(C, D) * 0.1 # Small random values

    # Initialize covariances: S_k S_k^T + I_D
    covariances = np.zeros((C, D, D))
    for k in range(C):
        S_k = np.random.randn(D, D) # Gaussian random matrix
        covariances[k] = S_k @ S_k.T + np.eye(D)

    log_likelihoods = []
    prev_log_likelihood = -np.inf

    for iteration in range(max_iter):
        # E-Step: Calculate responsibilities (tau_k_i)
        tau_k_i = np.zeros((N, C))
        for k in range(C):
            pdf_values = gaussian_pdf(X, means[k], covariances[k])
            tau_k_i[:, k] = weights[k] * pdf_values

        # Normalize responsibilities for each data point
        sum_tau = np.sum(tau_k_i, axis=1, keepdims=True)
        # Handle cases where sum_tau might be zero (very low probability data point
        # Add a small epsilon to avoid division by zero
        tau_k_i /= (sum_tau + 1e-300)

        # M-Step: Update parameters
        # Update weights
        N_k = np.sum(tau_k_i, axis=0)
        weights = N_k / N

        # Update means
        for k in range(C):
            means[k] = np.sum(tau_k_i[:, k, np.newaxis] * X, axis=0) / (N_k[k] + 1e

        # Update covariances
        for k in range(C):
            X_minus_mean_k = X - means[k]
            # Reshape tau_k_i[:, k] to (N, 1) for broadcasting
            covariances[k] = (X_minus_mean_k.T @ (tau_k_i[:, k, np.newaxis] * X_min
            # Add a small value to the diagonal for numerical stability (regulariza
            covariances[k] += np.eye(D) * 1e-6
```

```
        # Calculate log-likelihood
        current_log_likelihood = calculate_log_likelihood(X, weights, means, covari
        log_likelihoods.append(current_log_likelihood)

        # Check for convergence
        if iteration > 0 and abs(current_log_likelihood - prev_log_likelihood) < to
            print(f"EM converged at iteration {iteration}")
            break
        prev_log_likelihood = current_log_likelihood

        # Optional: Print progress
        if (iteration + 1) % 10 == 0 or iteration == 0:
            print(f"Iteration {iteration+1}, Log-Likelihood: {current_log_likelihoo
```

In [ ]:
```
# Run EM algorithm
C = 2 # Number of components for GMM (for digits '2' and '6')
weights, means_pca, covariances_pca, responsibilities, log_likelihoods = em_gmm(X_p

# Plot Log-Likelihood
plt.figure(figsize=(10, 6))
plt.plot(range(len(log_likelihoods)), log_likelihoods, marker='o', linestyle='-')
plt.title('Log-Likelihood vs. Number of Iterations')
plt.xlabel('Iteration')
plt.ylabel('Log-Likelihood')
plt.grid(True)
plt.show()
```

In [ ]:
```
# Part 2: Report Fitted GMM Model

print("--- Fitted GMM Model Parameters ---")

# a) Numerical weights for each component
print("\n1. Component Weights:")
for i, w in enumerate(weights):
    print(f"   Component {i+1}: {w:.4f}")

# b) Mean of each component (mapped back to original space and as 28x28 images)
print("\n2. Component Means (as 28x28 Images):")

# Map means back to original 784-dimensional space
# The inverse transform requires the components and mean from PCA
# X_original_recon = pca.inverse_transform(X_pca) would give the full data reconstr
# To reconstruct the mean of a component, we use pca.inverse_transform on the mean
# Note: The pca.mean_ attribute is crucial here for correct reconstruction
original_space_means = pca.inverse_transform(means_pca)

plt.figure(figsize=(10, 5))
for i, mean_vec in enumerate(original_space_means):
    # Reshape the 784-dimensional vector to 28x28 image
    mean_image = mean_vec.reshape(28, 28)

    plt.subplot(1, C, i + 1)
    plt.imshow(mean_image, cmap='gray')
    plt.title(f'Component {i+1} Mean (Digit {2 if i==0 else 6})' if labels_2_6[resp
    plt.axis('off')
plt.suptitle('Mean Images of GMM Components (in Original 784-dim Space)')
plt.show()


# c) Two 4x4 covariance matrices (visualized as heatmaps)
print("\n3. Covariance Matrices (4x4 Heatmaps):")
plt.figure(figsize=(12, 5))
for i, cov_mat in enumerate(covariances_pca):
    plt.subplot(1, C, i + 1)
    plt.imshow(cov_mat, cmap='viridis', interpolation='nearest')
    plt.colorbar(label='Covariance Value')
    plt.title(f'Component {i+1} Covariance Matrix')
    plt.xlabel('Dimension')
    plt.ylabel('Dimension')
plt.suptitle('Covariance Matrices of GMM Components (in 4-dim PCA Space)')
plt.show()

print("\nNumerical Covariance Matrices:")
for i, cov_mat in enumerate(covariances_pca):
    print(f"\nComponent {i+1} Covariance Matrix:\n{cov_mat.round(4)}")
```

In [ ]:

```
# Part 3: Infer Labels and Compare with True Labels

# Determine which GMM component corresponds to '2' and '6'
# We assign the component label based on the majority true label of the data points
cluster_labels_map = {}
for k in range(C):
    # Find the indices of data points primarily assigned to component k
    component_k_indices = np.where(np.argmax(responsibilities, axis=1) == k)[0]
    if len(component_k_indices) > 0:
        # Get the true labels for these data points
        true_labels_in_component_k = labels_2_6[component_k_indices]
        # Count occurrences of '2' and '6'
        count_2 = np.sum(true_labels_in_component_k == 2)
        count_6 = np.sum(true_labels_in_component_k == 6)

        # Assign the cluster label based on the majority
        if count_2 > count_6:
            cluster_labels_map[k] = 2
        else:
            cluster_labels_map[k] = 6
    else:
        # If no data points are assigned, assign a default or handle as an edge cas
        cluster_labels_map[k] = -1 # Or some other indicator

print("GMM Cluster to True Label Mapping:", cluster_labels_map)

# Infer GMM labels for all data points
gmm_predicted_raw_labels = np.argmax(responsibilities, axis=1)
gmm_predicted_labels = np.array([cluster_labels_map[label] for label in gmm_predict

# Calculate overall accuracy and mis-classification rate for GMM
correct_predictions_gmm = np.sum(gmm_predicted_labels == labels_2_6)
total_predictions = len(labels_2_6)
accuracy_gmm = correct_predictions_gmm / total_predictions
misclassification_rate_gmm = 1 - accuracy_gmm

print(f"\n--- GMM Performance ---")
print(f"GMM Overall Accuracy: {accuracy_gmm:.4f}")
print(f"GMM Overall Mis-classification Rate: {misclassification_rate_gmm:.4f}")

# Calculate mis-classification rate for '2' and '6' separately for GMM
mis_2_gmm = np.sum((gmm_predicted_labels != 2) & (labels_2_6 == 2)) / np.sum(labels
mis_6_gmm = np.sum((gmm_predicted_labels != 6) & (labels_2_6 == 6)) / np.sum(labels

print(f"GMM Mis-classification Rate for Digit '2': {mis_2_gmm:.4f}")
print(f"GMM Mis-classification Rate for Digit '6': {mis_6_gmm:.4f}")


# Perform K-Means clustering with K=2
print("\n--- K-Means Clustering Performance ---")
kmeans = KMeans(n_clusters=C, random_state=0, n_init=10) # n_init for robust initia
kmeans_labels_raw = kmeans.fit_predict(X_pca)

# Map K-Means clusters to true labels ('2' or '6')
kmeans_cluster_labels_map = {}
for k in range(C):
    component_k_indices = np.where(kmeans_labels_raw == k)[0]
    if len(component_k_indices) > 0:
```

```
        true_labels_in_component_k = labels_2_6[component_k_indices]
        count_2 = np.sum(true_labels_in_component_k == 2)
        count_6 = np.sum(true_labels_in_component_k == 6)
        if count_2 > count_6:
            kmeans_cluster_labels_map[k] = 2
        else:
            kmeans_cluster_labels_map[k] = 6
    else:
        kmeans_cluster_labels_map[k] = -1

print("K-Means Cluster to True Label Mapping:", kmeans_cluster_labels_map)

kmeans_predicted_labels = np.array([kmeans_cluster_labels_map[label] for label in k

# Calculate overall accuracy and mis-classification rate for K-Means
correct_predictions_kmeans = np.sum(kmeans_predicted_labels == labels_2_6)
accuracy_kmeans = correct_predictions_kmeans / total_predictions
misclassification_rate_kmeans = 1 - accuracy_kmeans

print(f"K-Means Overall Accuracy: {accuracy_kmeans:.4f}")
print(f"K-Means Overall Mis-classification Rate: {misclassification_rate_kmeans:.4f

# Calculate mis-classification rate for '2' and '6' separately for K-Means
mis_2_kmeans = np.sum((kmeans_predicted_labels != 2) & (labels_2_6 == 2)) / np.sum(
mis_6_kmeans = np.sum((kmeans_predicted_labels != 6) & (labels_2_6 == 6)) / np.sum(

print(f"K-Means Mis-classification Rate for Digit '2': {mis_2_kmeans:.4f}")
print(f"K-Means Mis-classification Rate for Digit '6': {mis_6_kmeans:.4f}")

print("\n--- Comparison ---")
if misclassification_rate_gmm < misclassification_rate_kmeans:
    print("GMM achieves better overall performance (lower mis-classification rate).
elif misclassification_rate_gmm > misclassification_rate_kmeans:
    print("K-Means achieves better overall performance (lower mis-classification ra
else:
    print("GMM and K-Means achieve similar overall performance.")
```

In [ ]:

In [ ]:
```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore") # Suppress warnings, especially from LogisticRegr
```

In [ ]:
```
# Load the dataset
try:
    df = pd.read_csv('marriage.csv')
    print("Dataset loaded successfully.")
except FileNotFoundError:
    print("Error: 'marriage.csv' not found. Please ensure it's in the same director
    # Create dummy data for demonstration if file is not found
    print("Generating dummy data for demonstration purposes.")
    num_participants = 170
    num_features = 54
    dummy_data = np.random.rand(num_participants, num_features - 1) * 5 # Simulate
    dummy_labels = np.random.randint(0, 2, num_participants).reshape(-1, 1) # Simul
    df = pd.DataFrame(np.hstack((dummy_data, dummy_labels)), columns=[f'feature_{i}
    print("Dummy data generated.")

# Separate features (X) and target (y)
X = df.iloc[:, :-1]  # All columns except the last one
y = df.iloc[:, -1]   # The last column (label)

print(f"Features (X) shape: {X.shape}")
print(f"Target (y) shape: {y.shape}")
print("\nFirst 5 rows of features (X):")
print(X.head())
print("\nFirst 5 values of target (y):")
print(y.head())
```

In [ ]:
```
# Split the dataset into 80% training and 20% testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta
# Using stratify=y ensures that the proportion of classes in the train and test set

print(f"X_train shape: {X_train.shape}")
print(f"X_test shape: {X_test.shape}")
print(f"y_train shape: {y_train.shape}")
print(f"y_test shape: {y_test.shape}")

print("\nDistribution of classes in y_train:")
print(y_train.value_counts(normalize=True))
print("\nDistribution of classes in y_test:")
print(y_test.value_counts(normalize=True))
```

In [ ]:
```
# Gaussian Naive Bayes Classifier
print("--- Gaussian Naive Bayes ---")

# Custom variance handling for Naive Bayes (as per remark)
# We will estimate variances from training data and then adjust
epsilon = 1e-3

# Calculate variance for each feature in X_train
variances = X_train.var()

# Identify features with variance close to zero and adjust
# Note: GaussianNB has a `var_smoothing` parameter, but the problem implies manual
# If `var_smoothing` is used, it adds this value to the variance for numerical stab
# Here, we are explicitly checking and adjusting zero/near-zero variances before pa
# However, scikit-learn's GaussianNB handles this internally with `var_smoothing` b
# If we were to implement Naive Bayes from scratch, we'd apply this.
# For scikit-learn, the simplest way to strictly follow "set variance to small numb
# is to manually calculate parameters or preprocess X_train.
# Let's check how GaussianNB internally estimates variance. It typically adds a `va
# term. The prompt's wording "if the variance is zero... set the variance to be a s
# suggests a more direct intervention on the estimated variances.

# A common way to implement the spirit of the remark for scikit-learn's GaussianNB
# is to apply var_smoothing, or directly modify the X_train by adding noise if vari
# However, modifying X_train can be complex. The simplest direct application to Gau
# is via `var_smoothing`. Let's use it as it's designed for this purpose.
# If explicit manual variance calculation and replacement is required, that would m
# reimplementing GaussianNB likelihood computations. For now, var_smoothing is the
# If the intention was to specifically estimate variances, modify them, and then pa
# to a custom Naive Bayes implementation, that's a different task.
# Given the use of `scikit-learn` for other classifiers, `var_smoothing` is the mos
# direct interpretation of the requirement for `GaussianNB`.

gnb = GaussianNB(var_smoothing=epsilon) # Adds epsilon to the variance of each feat
gnb.fit(X_train, y_train)
y_pred_gnb = gnb.predict(X_test)

print(f"Accuracy: {accuracy_score(y_test, y_pred_gnb):.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred_gnb))
print("\nConfusion Matrix:")
cm_gnb = confusion_matrix(y_test, y_pred_gnb)
print(cm_gnb)

# Plot Confusion Matrix
plt.figure(figsize=(6, 5))
sns.heatmap(cm_gnb, annot=True, fmt='d', cmap='Blues', xticklabels=['Predicted No D
plt.title('Confusion Matrix - Gaussian Naive Bayes')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```

In [ ]:
```
# Logistic Regression Classifier
print("\n--- Logistic Regression ---")

# Using default solver 'lbfgs' or 'liblinear' often works well.
# 'liblinear' is good for small datasets and L1/L2 regularization.
# 'lbfgs' is a good default. Max_iter increased for convergence.
log_reg = LogisticRegression(random_state=42, solver='liblinear', max_iter=1000)
log_reg.fit(X_train, y_train)
y_pred_log_reg = log_reg.predict(X_test)

print(f"Accuracy: {accuracy_score(y_test, y_pred_log_reg):.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred_log_reg))
print("\nConfusion Matrix:")
cm_log_reg = confusion_matrix(y_test, y_pred_log_reg)
print(cm_log_reg)

# Plot Confusion Matrix
plt.figure(figsize=(6, 5))
sns.heatmap(cm_log_reg, annot=True, fmt='d', cmap='Blues', xticklabels=['Predicted
plt.title('Confusion Matrix - Logistic Regression')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```

In [ ]:
```
# K-Nearest Neighbors Classifier
print("\n--- K-Nearest Neighbors (KNN) ---")

# Choosing n_neighbors: A common practice is sqrt(N) or odd numbers.
# For N=170, sqrt(170) is approx 13. Let's try 5 as a starting point.
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
y_pred_knn = knn.predict(X_test)

print(f"Accuracy: {accuracy_score(y_test, y_pred_knn):.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred_knn))
print("\nConfusion Matrix:")
cm_knn = confusion_matrix(y_test, y_pred_knn)
print(cm_knn)

# Plot Confusion Matrix
plt.figure(figsize=(6, 5))
sns.heatmap(cm_knn, annot=True, fmt='d', cmap='Blues', xticklabels=['Predicted No D
plt.title('Confusion Matrix - K-Nearest Neighbors (KNN)')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```

In [ ]:
```
print("\n--- Summary of Classifier Performance ---")
print(f"Gaussian Naive Bayes Accuracy: {accuracy_score(y_test, y_pred_gnb):.4f}")
print(f"Logistic Regression Accuracy:  {accuracy_score(y_test, y_pred_log_reg):.4f}
print(f"K-Nearest Neighbors (KNN) Accuracy: {accuracy_score(y_test, y_pred_knn):.4f

# You can also compare F1-scores, precision, recall etc.
# For simplicity, let's just stick to accuracy for a quick comparison.
```

In [ ]: